无重叠区间问题

一、问题分析

给定一个由 n 个区间组成的集合 intervals，每个区间表示为 $[start_i, end_i]$,(端点接触不算重叠)。要求移除最少数量的区间，使得剩下的区间两两互不重叠。题目等价于：选出尽可能多的互不重叠的区间，然后返回 n 减去该最大数量。

（1）输入：二维向量 intervals，每个元素是长度为 2 的向量。

（2）输出：需要移除的最小区间数。

（3）边界条件：若输入为空，直接返回 0。

（4）重叠定义：区间 [a,b] 与 [c,d] 重叠，当且仅当 $a < d$ 且 $c < b$；若仅端点接触（如 $b = c$），不算重叠。因此判断条件为：后一区间的起点 ≥ 前一区间的终点，即不重叠。

二、贪心策略

贪心选择：

总是选择结束时间最早的区间，然后排除所有与之重叠的区间，在剩余兼容区间中重复这一过程，直到遍历完所有区间。

具体操作：

1. 将全部区间按照结束时间升序排序。
2. 初始化：选择第一个区间（结束时间最早），记录其结束时间 last_end，已选不重叠区间计数 count = 1。
3. 依次遍历后续区间：

   （1）如果当前区间的起点 ≥ last_end，则说明它与前一个选中区间不重叠，将其选中，更新 last_end 为当前区间的结束点，count 加 1。

   （2）否则，该区间与上一个选中区间重叠，必须移除（不做计数，直接跳过）。
4. 最终，返回需要移除的数量 = $n - count$。

三、正确性证明

3.1 最优子结构性质

设区间集合为 S，按结束时间升序排序后，假设第一个区间（结束时间最早）为 i_1。若最优解包含 i_1，那么剩下的问题是在所有与 i_1 不重叠的区间子集 $S' \subseteq S$ 中求最大不重叠区间数。原问题的最优解必定由 i_1 和子问题 S' 的最优解组成（可通过反证法证明）。这说明问题具有最优子结构：最大不重叠区间数问题可以分解为选择当前区间后的子问题，且子问题的解合并后仍是最优的。

3.2 贪心选择性质

命题：对于按结束时间排序后的区间集合，存在一个最大不重叠区间子集，它包含结束时间最早的区间（即第一个区间）。

证明：

所有区间按结束时间升序排列为 $I_1, I_2, \ldots, I_n$，其中 $end_1 \leq end_2 \leq \cdots \leq end_n$。设 OPT 为任意一个最大不重叠区间子集，并按结束时间递增排序，设其第一个区间为 I_k。

（1）若 $k = 1$，则 OPT 已经包含 I_1，结论成立。

（2）若 $k > 1$，由于 I_1 是结束时间最早的区间，有 $end_1 \leq end_k$。构造一个新的集合 $OPT' = (OPT \setminus \{I_k\}) \cup \{I_1\}$。因为 I_k 是 OPT 中开始时间最早的区间，且 OPT 中其他区间均始于 I_k 结束之后（$start \geq end_k$），而 $end_1 \leq end_k$，所以 I_1 的结束时间不晚于 I_k，因此 I_1 也不可能与 OPT 中除 I_k 外的任何区间重叠。替换后，OPT' 依然是不重叠区间集合，且大小与 OPT 相同，仍为最大集合。因此存在一个包含 I_1 的最优解。

由此，贪心选择（每次选取结束时间最早的兼容区间）能逐步构造出全局最优解。结合最优子结构和贪心选择性质，该贪心算法正确。

四、算法设计与伪代码

算法：eraseOverlapIntervals(intervals)

输入：区间列表 intervals，每个区间 [start, end]

输出：需要移除的最小区间数量

```
1.  if intervals 为空 then
2.      return 0
3.  end if
4.  将 intervals 按照区间结束时间 end 升序排序
```

```
5.  count ← 1                                  // 至少可以选择第一个区间
6.  last_end ← intervals[0][1]       // 记录最后一个选中区间的结束时间
7.  for i ← 1 to |intervals| - 1 do
8.          if intervals[i][0] ≥ last_end then
9.                  count ← count + 1
10.                 last_end ← intervals[i][1]
11.         end if
12. end for
13. return |intervals| - count
```

五、时空复杂性分析

1. 时间复杂度：

（1）排序：按结束时间排序需要 $O(n\log n)$ 时间。

（2）遍历：单次线性扫描，比较和更新操作均为常数时间，需要 $O(n)$ 时间。

（3）总时间复杂度为 $O(n\log n)$，其中 n 为区间个数。

2. 空间复杂度：

（1）排序通常需要 $O(\log n)$ 的栈空间（如快速排序递归深度）。

（2）除输入和排序所需的空间外，只使用了常数个辅助变量（count、last_end、循环变量），额外空间为 $O(1)$。

（3）总空间复杂度为$O(\log n)$，其中 n 为区间个数。

最大多位整数问题

一、问题分析

给定 n 个正整数，要求将它们连接成一排，组成一个最大的多位整数。本质上是要寻找一种排列顺序，使得按此顺序拼接得到的字符串对应的数值最大。

（1）输入：n 个正整数（可用数组或向量表示）。

（2）输出：拼接后的最大整数，以字符串形式返回（避免大数溢出）。

（3）边界条件：若所有数字均为 0，结果应为 "0" 而非 "00...0"；若输入为空，

返回空串。

（4）比较依据：对于两个数字串 a 与 b，a 排在 b 之前能产生更大拼接结果的条件是 $a+b>b+a$（字符串拼接后按字典序比较）。

二、贪心策略

贪心选择：

每次都想让排在更高位的数字尽可能“大”，但此处的“大”不是单个数字的数值大小，而是对最终拼接结果的贡献大小。

具体操作：

（1）将所有整数转化为字符串。

（2）定义自定义比较规则：若字符串 a 与 b 满足 $a+b>b+a$，则 a 应排在 b 之前。

（3）使用该规则对所有字符串进行降序排序。

（4）将排序后的字符串依次拼接，得到答案。

（5）特例：如果排序后第一个字符串为 "0"，说明所有数字都是 0，此时直接返回 "0"，避免出现 "000" 之类的前导零情况。

三、正确性证明

3.1 最优子结构性质

设原问题为求数字集合 S 的最大拼接串，若在某个最优解中第一个数字为 x，则余下数字的排列顺序必须是子问题 $S\setminus\{x\}$ 的最优解。否则，假设剩下的排列不是该子问题的最优解，即存在另一个由 $S\setminus\{x\}$ 中的数字组成的排列 Q，使得 Q 的拼接结果严格大于原来余下部分的拼接结果；那么将 x 与 Q 拼接后，得到的完整数字串将严格大于原来的最优解，这与该解是最优的假设矛盾。因此，原问题的最优解必然由第一个数字 x 和从剩余数字构成的最优解组合而成，问题具有最优子结构性质。

3.2 贪心选择性质

设按比较规则（即若 $a+b>b+a$ 则 a 排在 b 前）排出的最靠前的数字为 x^*，即对任意数字 a 均有 $x^*+a\geq a+x^*$。现要证明存在一个最优排列以 x^* 开头。假设存在某个最优排列 P，它的第一个数字不是 x^*，而是某个 y。由于 x^* 是集合中的元素，它必然出现在 P 中的某个位置。我们从 x^* 所在的位置开始，反复

将 x^* 与其左侧相邻的数字进行交换，逐步向开头移动。在每一步中，设 x^* 左侧相邻的数字为 a，则交换前局部为 a, x^*，交换后变为 x^*, a；由于对任意 a 都满足 $x^* + a \geq a + x^*$，因此交换后的局部拼接结果不会小于交换前，而序列其余部分保持不变，因此每一次相邻交换都不会使整个序列的拼接结果变小。经过有限次这样的相邻交换后，x^* 被移动到序列的第一位，得到的新排列 P' 以 x^* 开头，且其拼接结果不小于原最优排列 P 的结果。由于 P 已经是最大值，故 P' 也是最优排列，这就证明了存在一个以 x^* 开头的最优解，贪心选择性质成立。

四、算法设计与伪代码

输入：正整数数组 nums

输出：表示最大整数的字符串

1. if nums 为空 then
2. return ""
3. end if
4. 将 nums 中每个整数转换为字符串，存入数组 ans
5. 按自定义规则对 ans 进行降序排序：

   规则：对任意 a, b ∈ sans，若 a + b > b + a，则 a 排在 b 前
6. if nums[0] == "0" then
7. return "0"
8. end if
9. result ← ""
10. for each s in ans do
11. result ← result + s
12. end for
13. return result

五、时空复杂性分析

1. 时间复杂度

（1）数值转字符串：$O(n \cdot M)$，其中 M 为数字的最大长度（位数）。

（2）排序：比较次数为 $O(n\log n)$，每次比较需要拼接字符串 a+b 和 b+a，长度至多为 $2M$，字典序比较耗时 $O(M)$。故排序总耗时 $O(n\log n \cdot M)$。

（3）拼接结果：$O(n \cdot M)$。

当 M 为常数时，时间复杂度可视为 $O(n\log n)$, 一般情况写作 $O(n\log n \cdot M)$。

2. 空间复杂度

（1）存储 n 个字符串需要 $O(n \cdot M)$ 空间。

（2）排序递归栈深度 $O(\log n)$，额外变量 $O(1)$。总空间复杂度为 $O(n \cdot M)$（若 M 为常数则为 $O(n)$）。

最优服务次序问题

一、问题分析

设有 n 个顾客同时到达并等待同一项服务，第 i 个顾客需要的服务时间为 t_i。服务必须依次进行，每个顾客的等待时间定义为从开始等待到其服务结束所经历的时间，但通常只计算排队等待时间（即前面所有顾客服务时间之和）。目标是通过安排服务次序，使所有顾客的平均等待时间最小。

（1）输入：正整数 n 以及长度为 n 的服务时间数组 time。

（2）输出：最小平均等待时间（保留小数）。

（3）边界条件：若 $n = 0$，平均等待时间可定义为 0。

（4）等待时间定义：若服务次序为 $p_1, p_2, \dots, p_n$，则第 p_k 个顾客的等待时间为 $\sum_{j=1}^{k-1} t_{p_j}$，总等待时间为 $\sum_{k=1}^{n} \sum_{j=1}^{k-1} t_{p_j}$，平均等待时间为总等待时间除以 n。

二、贪心策略

贪心选择：**总是选择当前服务时间最短的顾客优先服务**。

具体操作：

（1）将所有顾客按服务时间升序排序。

（2）按排序后的顺序依次服务：第一个顾客等待 0，第二个顾客等待第一个顾客的服务时间，第三个顾客等待前两个顾客的服务时间之和，依此类推。

（3）累加所有等待时间得到总等待时间，再除以 n 得到平均等待时间。

三、正确性证明

3.1 最优子结构性质

设 S 为全体顾客集合，其服务时间分别为 $t_1, t_2, \dots, t_n$，并设 S 的一个最优服务次序中第一个接受服务的顾客为 i，服务时间为 t_i。在该次序下，第一个顾客的等待时间为 0，其余 $n-1$ 个顾客每人都必须等待第一个顾客的服务完成，因此这 $n-1$ 个顾客的等待时间均包含一个 t_i。总等待时间可分解为 $T = (n-1) \cdot t_i + T'$，其中 T' 是剩余 $n-1$ 个顾客按照该次序中后续排列进行服务时产生的总等待时间。若剩余 $n-1$ 个顾客存在另一种排列使得其总等待时间 $T'' < T'$，则用该排列替换原次序中后续部分，得到的新次序总等待时间为 $(n-1) \cdot t_i + T'' < T$，与原次序的最优性矛盾。因此，在原最优次序中，余下 $n-1$ 个顾客的排列必须构成子问题 $S \setminus \{i\}$ 的最优解，原问题的最优解由第一个顾客 i 与子问题的最优解合并而成，问题具有最优子结构性质。

3.2 贪心选择性质

命题：存在一个最优服务次序，其第一个顾客是所有顾客中服务时间最短的那一个。

证明：

设全体顾客中服务时间的最小值为 $t_{\min}$，其对应顾客为 m。任取一个最优服务次序 $S = (a_1, a_2, \dots, a_n)$，若 $a_1 = m$，则结论已成立。若 $a_1 \neq m$，设 m 在 S 中的位置为 k（$k > 1$），即 $a_k = m$，且 $t_{a_k} = t_{\min} < t_{a_1}$。构造新次序 S'：将 a_1 与 a_k 交换位置，其余顾客相对顺序不变。此时前 k 个顾客的等待时间变化如下：在原次序中，前 k 人的等待时间之和为 $(k-1)t_{a_1} + (k-2)t_{a_2} + \cdots + 1 \cdot t_{a_{k-1}}$；在新次序中，前 k 人变为 $a_k, a_2, a_3, \dots, a_{k-1}, a_1$，其等待时间之和为 $(k-1)t_{a_k} + (k-2)t_{a_2} + \cdots + 1 \cdot t_{a_{k-1}}$。二者相减，前 k 人的总等待时间减少量为 $\big(k-1)(t_{a_1} - t_{a_k}\big)$。对于位置 $k+1$ 及之后的顾客，由于前 k 个顾客的服务时间总和在交换前后保持不变，故这些顾客的等待时间完全不受影响。因此，整个序列的总等待时间严格减少了 $(k-1)(t_{a_1} - t_{a_k}) > 0$，这意味着 S' 的总等待时间小于原最优次序 S，与 S 的最优性矛盾。故假设不成立，任意最优次序的第一个顾客必须具有最小的服务时间，贪心选择性质成立。

四、算法设计与伪代码

输入：服务时间数组 times

输出：最小平均等待时间

```
n ← length(times)
if n = 0 then
        return 0.0
end if
将 times 按升序排序
total_wait ← 0
cur_wait ← 0
for i ← 1 to n - 1 do
        cur_wait ← cur_wait + times[i-1]
        total_wait ← total_wait + cur_wait
end for
avg ← total_wait / n
return avg
```

五、时空复杂性分析

1. 时间复杂度：排序需要 $O(n\log n)$，后续线性累加需要 $O(n)$，总时间复杂度为 $O(n\log n)$。

2. 空间复杂度：排序递归栈深度 $O(\log n)$，只使用了常数个辅助变量，额外空间为 $O(1)$；若考虑输入数组的存储则为 $O(n)$。总空间复杂度可视为 $O(\log n)$ 或 $O(n)$。

Kruskal 解决最小生成树问题

一、问题分析

给定一个连通无向带权图 $G=(V,E)$，其中 V 为顶点集，E 为边集，每条边 (u,v) 有一个非负权重 $w(u,v)$。要求找出一个边子集构成生成树，使得所有顶点连通且无环，同时边权之和最小，即最小生成树。

（1）输入：顶点数 n，边列表 edges，每条边包含端点 u,v 和权重 w。

（2）输出：最小生成树的总权重。

（3）边界条件：若图为空或顶点数为 0，生成树权重为 0；若图不连通，则不存在生成树（通常题目保证连通）。

（4）性质：树包含 $n-1$ 条边，且无环。

二、贪心策略

贪心选择：每次从剩余边中选择一条权重最小且连接两个不同连通分量的边，将其加入最小生成树，同时合并这两个连通分量的集合，重复此过程直到选出 $n-1$ 条边为止。

具体操作：

（1）将所有边按权重升序排序。

（2）使用并查集维护顶点之间的连通关系。

（3）依次遍历排序后的边：若当前边连接的两个顶点不在同一集合中，则将该边加入生成树，累加权重，合并两顶点所在集合，并将已选边计数加 1。

（4）当边计数达到 $n-1$ 时停止。最终累加的权重即为最小生成树总权重。

三、正确性证明

3.1 最优子结构性质

设原图为 $G=(V,E)$，并设 Kruskal 算法第一步选中的边为 $e_1=(u,v)$，其权重 $w(e_1)$ 为全局最小。假设 T 为原图 G 的任意一棵最小生成树且包含 e_1。现在将 e_1 的两个端点 u 和 v 收缩为一个新顶点 uv，得到收缩后的图 G'：收缩时保留所有与 u 或 v 关联但不与 e_1 平行的边，若出现重边则仅保留权最小的一条。考虑 T 在 G' 中的对应边集 $T'=T\setminus\{e_1\}$，则 T' 构成 G' 的一棵生成树，且其总权重为 $w(T)-w(e_1)$。若 T' 不是 G' 的最小生成树，即存在另一棵生成树 T'' 满足 $w(T'')<w(T')$，则令 $T^*=T''\cup\{e_1\}$，它将构成原图 G 的一棵生成树，且 $w(T^*)=w(T'')+w(e_1)<w(T')+w(e_1)=w(T)$，与 T 是 G 的最小生成树矛盾。因此，T' 必为收缩后子图 G' 的最小生成树。这说明原问题的最优解可以由 e_1 与子问题（收缩后的图）的最优解合并而成，问题具有最优子结构性质。

3.2 贪心选择性质

命题：对于加权连通无向图 G，存在一棵最小生成树包含 Kruskal 算法按序选出的每一条边。

证明：先证第一条边 e_1（全局最小权边）必含于某棵最小生成树。设 T 是一棵最小生成树，若 $e_1 \in T$ 则已证。否则，将 e_1 加入 T 将形成一个唯一的环 C。在环 C 上，必存在一条边 $e' \neq e_1$。由于 e_1 是全局最小权边，有 $w(e_1) \leq w(e')$。考虑新边集 $T' = (T \setminus \{e'\}) \cup \{e_1\}$，它仍然连通且无环，从而是一棵生成树，且 $w(T') = w(T) - w(e') + w(e_1) \leq w(T)$。由于 T 是最小生成树，等号必须成立，故 T' 也是一棵最小生成树，且包含 e_1。至此基始得证。对于 Kruskal 算法后续选出的每一条边，算法实际上是在不断将两端点已连通分量视为同一顶点的收缩图上运行，因此每一步均可递归地应用上述论证于当前收缩图，从而存在一棵最小生成树包含该步所选边。由归纳法，算法最终选出的边集构成原图的一棵最小生成树，贪心选择性质成立。

四、算法设计与伪代码

算法：KruskalMST(edges, n)

输入：边列表 edges，每条边包含 (u, v, w)；顶点数 n
输出：最小生成树总权重

1. 将 edges 按权重 w 升序排序
2. 初始化并查集，每个顶点自成一个集合
3. total_weight ← 0
4. edge_count ← 0
5. for each e in edges do
6. if find(e.u) ≠ find(e.v) then
7. unite(e.u, e.v)
8. total_weight ← total_weight + e.w
9. edge_count ← edge_count + 1
10. end if
11. if edge_count = n - 1 then
12. break
13. end if
14. end for
15. return total_weight

五、时空复杂性分析

1. 时间复杂度：

边排序需要 $O(|E|\log|E|)$；并查集操作（含路径压缩与按秩合并）的均摊复杂度近乎常数，记为 $O(\alpha(|V|))$，其中 α 为反阿克曼函数。算法最多遍历所有边，因此总时间复杂度为 $O(|E|\log|E|+|E|\alpha(|V|))$，由于排序主导，可简化为 $O(|E|\log|E|)$。当图为稠密图时，$|E|$ 可达 $O(|V|^2)$，复杂度则为 $O(|V|^2\log|V|)$；一般表示为 $O(m\log m)$，m 为边数。

2. 空间复杂度：

存储边集需要 $O(|E|)$ 空间；并查集父节点数组需要 $O(|V|)$ 空间；排序递归栈深度 $O(\log|E|)$。总空间复杂度为 $O(|V|+|E|)$，通常简记为 $O(|E|)$。